TASK 13.02

Write the declaration of a record type to store the details of a book: Title, Year of publication, Price, ISBN.

Write the statements required to assign the values "Computer Science", 2019, £44.95, "9781108733755" to the fields respectively.

13.03 Arrays

Sometimes we want to organise data values into a list or a table / matrix. In most programming languages these structures are known as arrays. An array is an ordered set of data items, usually of the same type, grouped together using a single identifier. Individual array elements are addressed using an **array index** for each array dimension.

A list is a one-dimensional (1D) array and a table or matrix is a two-dimensional (2D) array.

TIP

When writing pseudocode, arrays need to be declared before they are used. This means choosing an identifier, the data type of the values to be stored in the array and **upper bound** and **lower bound** for each dimension.

KEY TERMS

Array index: row or column number of an individual array element

Upper bound: the highest number index of an array dimension

Lower bound: the smallest number index of an array dimension

13.04 One-dimensional arrays

When we write a list on a piece of paper and number the individual items, we would normally start the numbering with 1. You can view a 1D array like a numbered list of items. Many programming languages number array elements from 0 (the lower bound), including VB.NET, Python and Java. Depending on the problem to be solved, it might make sense to ignore element 0. The upper bound is the largest number used for numbering the elements of an array.

In pseudocode, a 1D array declaration is written as:

```
DECLARE <arrayIdentifier> : ARRAY[<lowerBound>:<upperBound>] OF <dataType>
```

Here is a pseudocode example:

```
DECLARE List1 : ARRAY[1:3] OF STRING  // 3 elements in this list
DECLARE List2 : ARRAY[0:5] OF INTEGER // 6 elements in this list
DECLARE List3 : ARRAY[1:100] OF INTEGER // 100 elements in this list
DECLARE List4 : ARRAY[0:25] OF CHAR  // 26 elements in this list
```

Accessing 1D arrays

A specific element in an array is accessed using an index value. In pseudocode, this is written as:

```
<arrayIdentifier>[x]
```

The *n*th element within the array `MyList` is referred to as `MyList[n]` .

Here is a pseudocode example:

```
NList[25] ← 0   // set 25th element to zero
AList[3] ← 'D' // set 3rd element to letter D
```

WORKED EXAMPLE 13.01

Working with a one-dimensional array

The problem to be solved: Take seven numbers as input and store them for later use.

We could use seven separate variables. However, if we wanted our algorithm to work with 70 numbers, for example, then setting up 70 variables would be complicated and waste time. Instead, we can make use of a data structure known as a 'linear list' or a 1D array.

This array is given an identifier, for example `MyList`, and each element within the array is referred to using this identifier and its position (index) within the array. For example, `MyList[4]` refers to the element at position 4 in the `MyList` array. If we are counting the element at position 0 as the first element, `MyList[4]` refers to the fifth element.

We can use a loop to access each array element in turn. If the numbers input to the pseudocode algorithm below are 25, 34, 98, 7, 41, 19 and 5 then the algorithm will produce the result in Figure 13.01.

```
FOR Index ← 0 TO 6
    INPUT MyList[Index]
NEXT Index
```

Index	MYList
[0]	25
[1]	34
[2]	98
[3]	7
[4]	41
[5]	19
[6]	5

Figure 13.01 `Mylist` array populated by a loop

TASK 13.03

Define two arrays, one for your friends' names and one for their ages as shown in Figure 13.02.

	Name
[0]	Matt
[1]	Fred
[2]	Anna
:	
:	
[19]	Xenios

	Age
[0]	15
[1]	16
[2]	14
:	
:	
[19]	17

Figure 13.02 Arrays for names and ages

WORKED EXAMPLE 13.02

Searching a 1D array

The problem to be solved: Take a number as input. Search for this number in an existing 1D array of seven numbers (see Worked Example 13.01).

Start at the first element of the array and check each element in turn until the search value is found or the end of the array is reached. This method is called a **linear search**.

Identifier	Data type	Explanation
`MyList`	`ARRAY[0:6] OF INTEGER`	Data structure (1D array) to store seven numbers
`MaxIndex`	`INTEGER`	The number of elements in the array
`SearchValue`	`INTEGER`	The value to be searched for
`Found`	`BOOLEAN`	`TRUE` if the value has been found `FALSE` if the value has not been found
`Index`	`INTEGER`	Index of the array element currently being processed

Table 13.02 Identifier table for linear search algorithm

```
MaxIndex ← 6
INPUT SearchValue
Found ← FALSE
Index ← -1
REPEAT
    Index ← Index + 1
    IF MyList[Index] = SearchValue
      THEN
        Found ← TRUE
    ENDIF
UNTIL FOUND = TRUE OR Index >= MaxIndex
IF Found = TRUE
  THEN
    OUTPUT "Value found at location: " Index
  ELSE
    OUTPUT "Value not found"
ENDIF
```

The complex condition to the `REPEAT...UNTIL` loop allows us to exit the loop when the search value is found. Using the variable `Found` makes the algorithm easier to understand. `Found` is initialised (first set) to `FALSE` before entering the loop and set to `TRUE` if the value is found.

If the value is not in the array, the loop terminates when `Index` is greater than or equal to `MaxIndex`. That means we have come to the end of the array. Note that using `MaxIndex` in the logic statement to terminate the loop makes it much easier to adapt the algorithm when the array consists of a different number of elements. The algorithm only needs to be changed in the first line, where `MaxIndex` is given a value.

KEY TERM

Linear search: checking each element of an array in turn for a required value

TASK 13.04

Use the algorithm in Worked Example 13.02 as a design pattern. Write an algorithm using the arrays from Task 13.03 to search for a friend's name and output their age.

WORKED EXAMPLE 13.03

Sorting elements in a 1D array

The simplest way to sort an unordered list of values is the following method.

1. Compare the first and second values. If the first value is larger than the second value, swap them.
2. Compare the second and third values. If the second value is larger than the third value, swap them.
3. Compare the third and fourth values. If the third value is larger than the fourth value, swap them.
4. Keep on comparing adjacent values, swapping them if necessary, until the last two values in the list have been processed.

Figure 13.03 shows what happens to the values as we work down the array, following this algorithm.

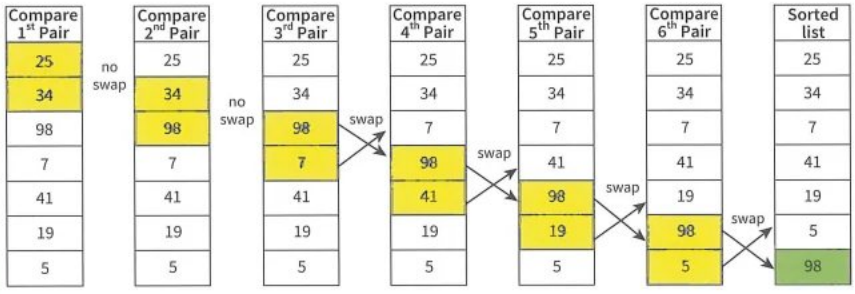


Figure 13.03 Swapping values working down the array

When we have completed the first pass through the entire array, the largest value is in the correct position at the end of the array. The other values may or may not be in the correct order.

We need to work through the array again and again. After each pass through the array the next largest value will be in its correct position, as shown in Figure 13.04.

(Continued)